

Git Documentation for QA Automation Projects

Git Documentation for QA Automation Projects

1	Introduction to Git Overview of Git and its importance
2	Git Setup & Configuration Steps to install and configure Git
3	Git Repository Initialization Creating or cloning a Git repository
4	Basic Git Commands Essential Git commands for daily use
5	Branching Strategy Strategies for managing branches in projects
6	Collaboration & Remote Repositories Working with remote repositories and collaboration
7	Resolving Merge Conflicts Handling conflicts during merging
8	Git Best Practices Best practices for using Git in QA automation
9	FAQ & Troubleshooting Common questions and troubleshooting tips

1. Introduction to Git

Git is a **distributed version control system (DVCS)** used to track changes in source code during software development. In **QA Automation**, Git helps:

- Maintain test scripts, frameworks, and configurations.
- Collaborate with team members efficiently.
- Track changes and revert if necessary.
- Integrate with CI/CD pipelines (Jenkins, GitHub Actions, etc.).

2. Git Setup & Configuration

Install Git

- **Windows:** Download from Git Official Site
- **Mac:** `brew install git`
- **Linux:** `sudo apt-get install git`

Configure Git

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
git config --global core.editor "code --wait" # (VS Code as default editor)
```

Check Configuration

```
git config --list
```

3. Git Repository Initialization

```
mkdir qa-automation-project
cd qa-automation-project
git init
```

Clone an Existing Repository

```
git clone <repository-url>
```

4. Basic Git Commands

CommandDescription

git status Check modified/staged files
git add <file> Stage a file for commit
git add . Stage all changes
git commit -m "message" Commit staged changes
git log View commit history
git diff See unstaged changes
git restore <file> Discard changes in a file
git rm <file> Remove a file from tracking

5. Branching Strategy for Automation Projects

Common Branches

- **main/master** → Production-ready automation scripts.
- **develop** → Integration branch for features.
- **feature/*** → New test cases/framework updates.
- **bugfix/*** → Fix failing tests.
- **hotfix/*** → Critical fixes in **main**.

Branch Commands

```
git branch # List branches
git branch <branch-name> # Create a new branch
git checkout <branch-name> # Switch branch
git checkout -b <branch-name> # Create & switch
git merge <branch-name> # Merge into current branch
git branch -d <branch-name> # Delete branch
```

6. Collaboration & Remote Repositories

Link Local to Remote (GitHub/GitLab/Bitbucket)

```
git remote add origin <repository-url>
git push -u origin <branch-name> # First push
git pull origin <branch-name> # Pull latest changes
git push origin <branch-name> # Push changes
```

Fork & Pull Request Workflow

1. Fork the repository (GitHub UI).
2. Clone your fork locally.
3. Make changes and push to your fork.
4. Create a **Pull Request (PR)** to the main repo.

7. Resolving Merge Conflicts

1. **Identify conflicts** (`git status` shows conflicts).
2. Open conflicted files and resolve manually [`<<<<<<, =====, >>>>>>`].
3. Mark as resolved:

```
git add <resolved-file>
git commit -m "Resolved merge conflict"
```

8. Git Best Practices for QA Automation

✔ **Commit small, logical changes** (e.g., "Add login test"). ✔ **Use descriptive commit messages.** ✔ **Follow a branching strategy.** ✔ **Pull before pushing** to avoid conflicts. ✔ **Use .gitignore** for logs, reports, and dependencies. ✔ **Leverage tags** for test suite versions (`git tag v1.0.0`).

Git Best Practices for QA Automation



9. FAQ & Troubleshooting

Q: How to undo a commit?

```
git reset --soft HEAD~1 # Keep changes, undo commit
git reset --hard HEAD~1 # Discard changes
```

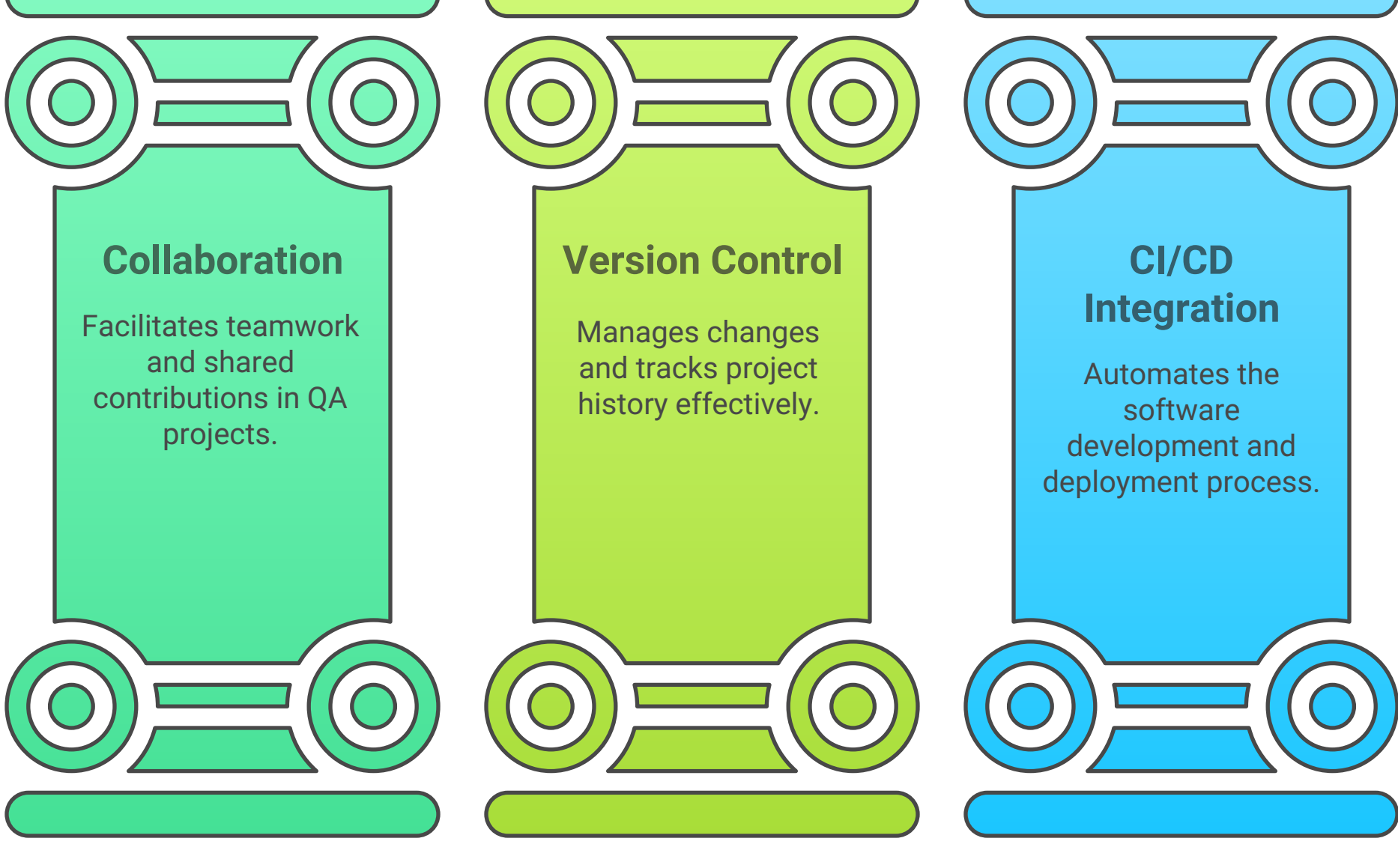
Q: How to sync a forked repo?

```
git remote add upstream <original-repo-url>
git fetch upstream
git merge upstream/main
```

Conclusion

This guide covers Git essentials for **QA Automation Architects**. Proper Git usage ensures smooth collaboration, version control, and CI/CD integration.

Mastering Git for Effective QA Automation Collaboration



Thank you